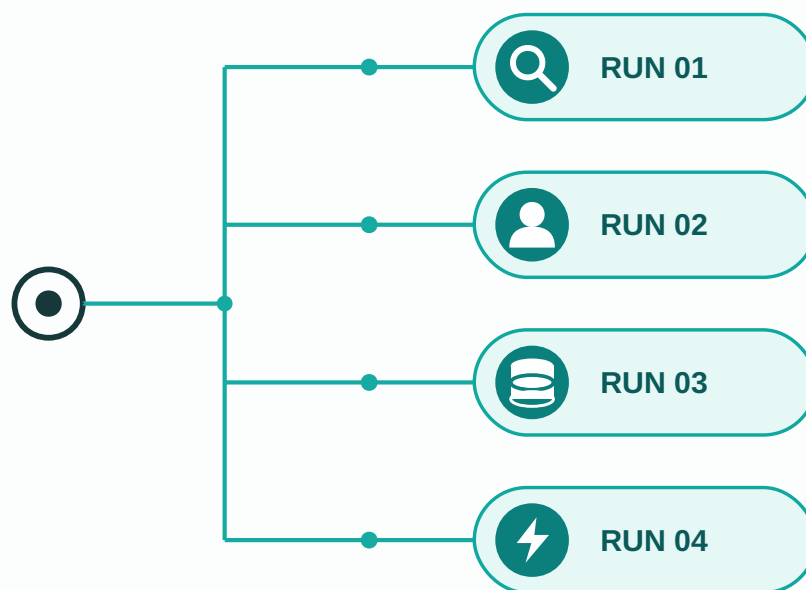


○ CHAPTER 01

LANGSMITH

Tracing, Observability & Evaluation for LLM Applications



WHAT THIS CHAPTER COVERS

- ▶ Why LangSmith: three real debugging scenarios
- ▶ Observability, defined — and what LangSmith traces
- ▶ Core concepts: Project, Trace and Run
- ▶ Tracing a simple LLM call & a sequential chain
- ▶ RAG tracing evolution: v1 partial trace → v4 cached pipeline
- ▶ Tracing an agent's tool-calling loop
- ▶ LangSmith + LangGraph integration
- ▶ Beyond observability: monitoring, evaluation & the LLMOps toolkit



Premium Revision Notes

LEVEL Intermediate–Advanced

FOCUS Observability · Tracing · Evaluation

FORMAT Study Notes



LangSmith — Observability, Tracing & Evaluation for LLM Applications

A detour from the LangGraph curriculum to learn how production LLM systems are debugged, monitored and evaluated. Assumes working knowledge of LangChain and the LangGraph fundamentals covered so far.

1. Why LangSmith? The Observability Problem

Every LLM-based system eventually reaches production, and every production system eventually breaks in a way its developer did not anticipate. Three recurring failure patterns motivate why a dedicated observability tool is needed for LLM applications, rather than relying on ordinary application logs.

1.1 Scenario One — A Latency Regression Nobody Can Explain

A startup builds an LLM application that helps students apply for jobs faster: the student pastes a job description, the tool studies it, fetches the student's resume and portfolio from Google Drive, matches relevant skills, drafts a cover letter, and proof-reads it. The whole pipeline normally takes about two minutes end-to-end. One day, users start complaining that the same request now takes seven to ten minutes.

The team cannot isolate the cause. They know the total input, the total output, and the total time — nothing else. Somewhere in the pipeline (perhaps a regression that made the app scan the student's entire Google Drive instead of one folder) a single stage is consuming most of the extra time, but with no per-component visibility there is no way to know which one.

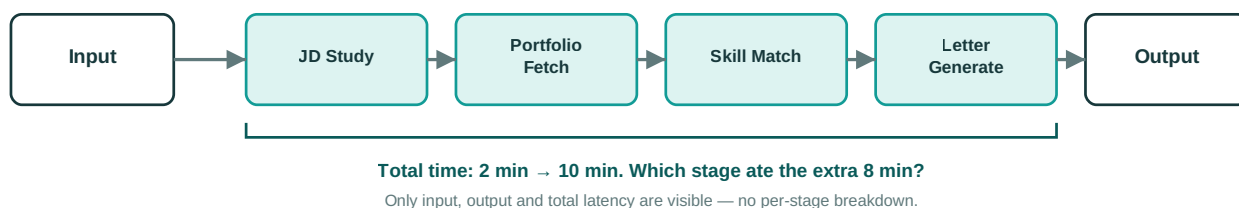


Figure 1.1 — Without observability, a multi-stage LLM workflow exposes only its overall input, output and total latency.

1.2 Scenario Two — A Cost Spike With No Clear Cause

A research-assistant agent fetches academic papers on a topic, extracts key points from each paper, and summarizes them into a report. It normally costs about ₹0.50 per report. After a prompt change that told the agent to "keep refining the report until it is as good as possible," some reports still cost ₹0.50 — but others silently cost ₹2 or more, because the agent decided to loop through the fetch–read–summarize cycle multiple times before it was satisfied.

Because agents are autonomous — they reason, act, and re-evaluate their own progress in a loop — a single small prompt edit can change behavior unpredictably across inputs. There is no consistent error, no crash, and no obvious log line; only an aggregate cost that occasionally spikes for reasons the team cannot see inside the agent's reasoning loop.

1.3 Scenario Three — A RAG Chatbot That Hallucinates

An internal HR chatbot answers employee questions (leave policy, notice period, insurance) using a knowledge base built from company documents — a standard Retrieval-Augmented Generation (RAG) design. It works well until it starts hallucinating: an employee asks about the leave policy and the bot invents a policy that does not exist, spreading misinformation inside the company.

A RAG system can hallucinate for two structurally different reasons, and the difference matters for debugging:

- **Retriever error** — the retriever fetches chunks that are not actually relevant to the question (for example, because k , the number of retrieved documents, was set too low).
- **Generator error** — the retrieved context is fine, but the LLM ignores it and answers from its own (possibly wrong) prior knowledge, often because the prompt does not strongly enough enforce "answer only from the given context."

Without visibility into what the retriever actually fetched and what the final assembled prompt actually contained, it is impossible to tell which of the two components is at fault.

IMPORTANT

All three scenarios share the same underlying difficulty: LLM systems are non-deterministic. The same input, or the same kind of input, can produce different outputs across runs. That non-determinism means failures rarely leave a clean stack trace — the system just behaves worse for some inputs than others, silently.

Quick Revision — Why Observability Matters

- Complex LLM workflows expose only input, output and total latency by default — never a per-stage breakdown.
- Agents can silently change cost or behavior after a minor prompt edit, because they reason and act in an autonomous loop.
- RAG hallucinations can originate from the retriever (wrong chunks) or the generator (ignoring correct chunks) — indistinguishable without visibility into intermediate steps.
- LLM non-determinism means the same input can produce different outputs, so failures rarely leave a traditional error trace.

1.4 Observability, Defined

Observability is the ability to understand a system's internal state by examining its external outputs — logs, metrics and traces — without needing to anticipate every failure mode in advance.

REAL WORLD INSIGHT

The core idea of observability is simple: instead of guessing what might go wrong and adding a log line for that specific case, you record everything a system does end-to-end, once, and it becomes possible to answer *why* something happened after the fact — even for a failure mode nobody anticipated.

Concretely, applying observability to an LLM application means: every time the application executes, that execution is traced end-to-end and the trace is stored somewhere it can be inspected later, component by component.

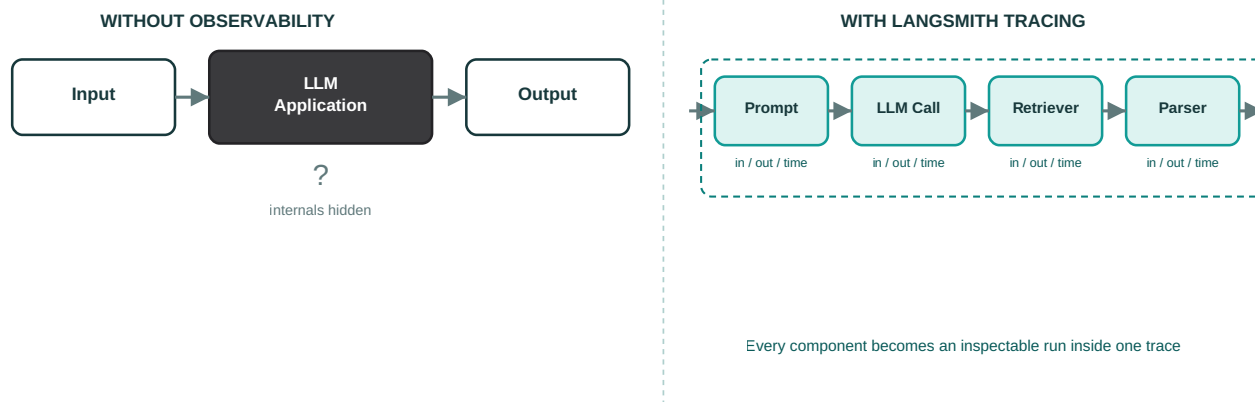


Figure 1.2 — Observability turns an opaque application into a component-by-component, inspectable trace.

1.5 LangSmith, Defined

LangSmith is a unified observability and evaluation platform where teams can debug, test and monitor AI application performance. In practice, LangSmith lets a developer run an LLM application and see the entire execution broken down component by component — what each component received as input, what it produced as output, and how long it took — at a very granular level.

1.6 What LangSmith Actually Traces

Before looking at any code, it helps to know upfront what LangSmith records for every execution it observes:

- The **input and output** of the overall application, plus every intermediate step (retriever query, retrieved context, assembled prompt, LLM output, parsed output).
- **Latency**, both for the whole application and for each individual component.
- **Token usage and cost**, based on the model being used, for input and output tokens separately.
- **Errors**, if any component fails.
- **Tags and metadata** — some auto-generated (for example, the exact model name), and some the developer attaches manually.
- **User feedback**, if the application chooses to attach it to a trace.

Quick Revision — Observability & LangSmith

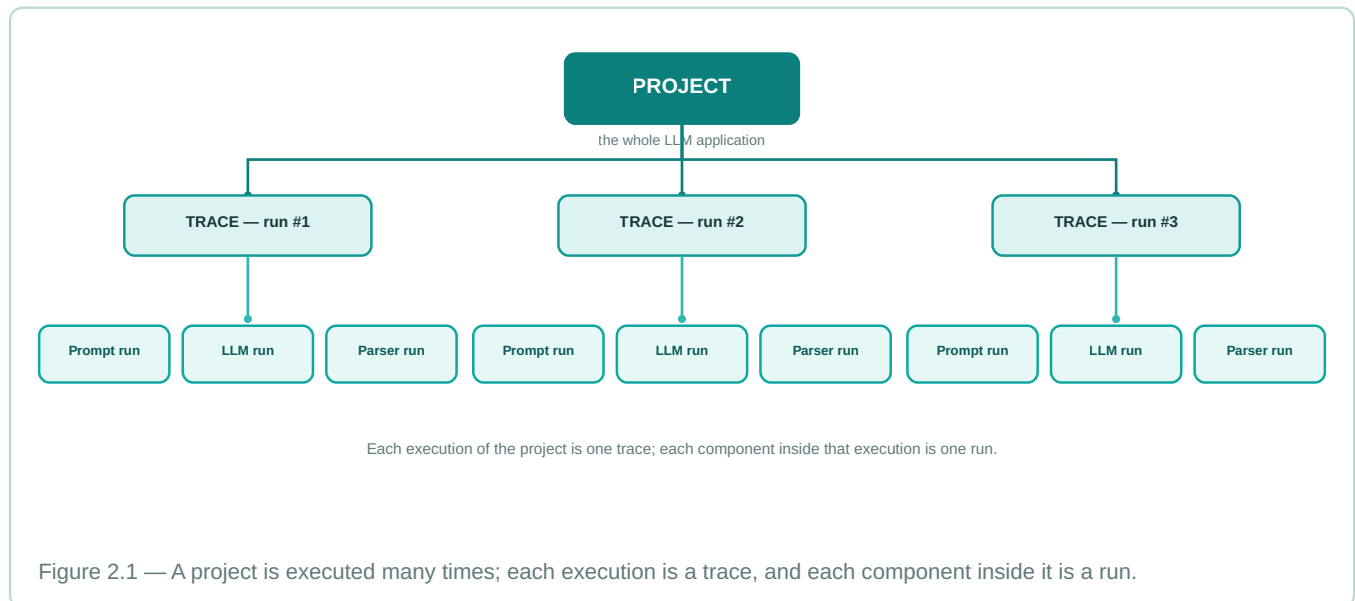
- Observability = understanding a system's internal state from its external outputs (logs, metrics, traces) without pre-anticipating failures.
- LangSmith is a unified observability and evaluation platform for debugging, testing and monitoring AI applications.
- LangSmith automatically captures input/output, intermediate steps, latency, token usage, cost, errors, tags and metadata for every traced execution.
- Tracing turns a black-box LLM workflow into a white-box, component-by-component view.

2. Core Concepts: Project, Trace, and Run

Before tracing anything, three vocabulary terms need to be pinned down precisely, because the rest of this chapter — and the LangSmith UI itself — is organized around them.

Consider the simplest possible LLM application: a user asks a question, the question is placed into a prompt, the prompt is sent to an LLM, and the LLM's reply is parsed and shown back to the user.

- **Project** — the entire LLM application, considered as a whole. Whatever you build (the simple Q&A app above, a RAG chatbot, an agent) becomes one project in LangSmith.
- **Trace** — one single end-to-end execution of a project. A project is executed many times — once per user request — and each such execution is one trace.
- **Run** — the execution of one individual component inside a trace. If the application is built from a prompt, an LLM and a parser, then a single trace of that application contains three runs: one for the prompt, one for the LLM, one for the parser.



IMPORTANT

These three terms nest inside each other in exactly one direction: a project contains many traces, and a trace contains many runs. A run never exists outside a trace, and a trace never exists outside a project. Keeping this hierarchy in mind makes the LangSmith UI far easier to navigate.

Quick Revision — Project / Trace / Run

- Project = the whole LLM application, as registered in LangSmith.
- Trace = one end-to-end execution of the project (one user request from start to finish).
- Run = the execution of one individual component (prompt, LLM call, retriever, parser, tool, graph node) inside a trace.
- Hierarchy is strict: Project → many Traces → many Runs.

2.1 Minimal Setup

Enabling LangSmith tracing for an existing LangChain or LangGraph application requires no code changes to the application logic itself — only a handful of environment variables. The demonstrations in this chapter use a project cloned from GitHub, a Python virtual environment, and the packages listed in `requirements.txt` (LangChain, LangGraph, LangSmith, Groq, FAISS, HuggingFace embeddings, and related tooling).

A LangSmith account provides a personal access key from **Settings** → **API Keys**. That key, together with the tracing endpoint and a flag to turn tracing on, goes into a `.env` file alongside the model provider's own API key:

```
OPENAI_API_KEY=<YOUR_API_KEY>

LANGCHAIN_TRACING_V2=true
LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
LANGCHAIN_API_KEY=<YOUR_LANGSMITH_API_KEY>
LANGCHAIN_PROJECT=langsmith-demo
```

BEST PRACTICE

`LANGCHAIN_PROJECT` decides which project a run's traces land under in the LangSmith UI. Give every distinct application its own project name — mixing an agent's traces into the same project as a simple chain's traces makes the trace list confusing to navigate later.

Once `LANGCHAIN_TRACING_V2` is set to `true` and a valid endpoint and key are present, LangChain and LangGraph runnables are traced automatically — no `import langsmith`, no decorators, no wrapper calls are required for ordinary chains.

3. Tracing a Simple LLM Call

The first and simplest demonstration is a one-step LLM application: a prompt template forwards a user's question directly to a chat model, and a string output parser extracts plain text from the model's reply.

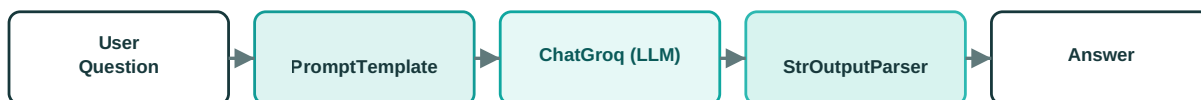


Figure 3.1 — Each stage of the simple chain — prompt, model, parser — becomes one run inside a single trace.

```
from dotenv import load_dotenv
from langchain_groq import ChatGroq
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# Simple one-line prompt
prompt = PromptTemplate.from_template("{question}")

model = ChatGroq(model = "openai/gpt-oss-20b")
parser = StrOutputParser()

# Chain: prompt → model → parser
chain = prompt | model | parser

result = chain.invoke({"question": "What is the capital of India?"})
print(result)
```

Nothing LangSmith-specific appears anywhere in this file. The moment `LANGCHAIN_TRACING_V2` is enabled in the environment, running this script automatically produces a trace under the `LANGCHAIN_PROJECT` configured in `.env` — here, `langsmith-demo` — with no further code changes.

TRACING PROJECT

LangSmith

Application

All applications

Search

Home

Tracing

Engine

Monitoring

Datasets & Experiments

Evaluators

Annotation Queues

Custom Apps

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

LLM Gateway

Settings

Personal

Personal / Tracing / langsmith-demo

langsmith-demo

Tracing

Evaluators

Automations

Insights

Engine

New

Retention 14d

Dashboard

+ New

Default View

Last 1 day

Threads

Traces

Runs

Input

Search input content

Add filter

<>

	✓	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost	First Token
	✓	RunnableSequence	What is the capital of India?	The capital of India is Ne...		9/1/2026, 103:...	0.50s			123	\$0.00001935	
	✓	RunnableSequence	What is the capital of Peru?	The capital of Peru is **LI...		9/1/2026, 102:...	0.61s			120	\$0.00001845	

The langsmith-demo project after two runs of the script — one question about India, one about Peru. Each row is a separate trace, with columns for input, output, latency, token count and cost already computed automatically.

Opening a project shows every trace produced by running the application — one row per execution. Clicking a trace opens a component-by-component breakdown: here, three runs corresponding to the three pipe-connected components — `PromptTemplate`, the `ChatGroq` LLM call, and `StrOutputParser` — each independently inspectable for its own input, output and latency.

REAL WORLD INSIGHT

LangSmith is intelligent enough to auto-tag a run with useful identifying metadata it can detect on its own — for example, the exact model name (`openai/gpt-oss-20b`) appears as a badge next to the LLM run without the developer specifying it. Library versions (LangChain core version, platform) are similarly captured automatically as run metadata.

Quick Revision — Simple LLM Call Tracing

- Tracing a LangChain runnable requires zero code changes — only the `LANGCHAIN_TRACING_V2` / endpoint / API key / project environment variables.
- A chain of `N` pipe-connected components (prompt | model | parser) produces `N` runs inside one trace.
- Each run shows its own input, output and latency; the trace as a whole shows total latency, total tokens and total cost.
- LangSmith auto-tags runs with detectable metadata such as the exact model name and library versions.

4. Tracing a Sequential Chain: Custom Projects, Tags, Metadata & Run Names

The second demonstration is a two-step sequential chain: a topic goes into a "generate a detailed report" prompt, the report comes back out and is fed into a second "generate a five-point summary" prompt, producing a final summary. This introduces four controls that go beyond the automatic tracing seen so far.

```
import os
from langchain_groq import ChatGroq
from dotenv import load_dotenv
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Project name set from inside the code (overrides the .env value)
os.environ['LANGCHAIN_PROJECT'] = 'Sequential LLM App'

load_dotenv()

prompt1 = PromptTemplate(template='Generate a detailed report on {topic}',
input_variables=['topic'])
prompt2 = PromptTemplate(template='Generate a 5 pointer summary from the following text \n
{text}', input_variables=['text'])

# 2. Two distinct models, explicitly named, with distinct temperatures
model1 = ChatGroq(model = "openai/gpt-oss-20b", temperature = 0.7)
model2 = ChatGroq(model = "openai/gpt-oss-20b", temperature = 0.5)
parser = StrOutputParser()

chain = prompt1 | model1 | parser | prompt2 | model2 | parser

# 3. Custom tags & metadata attached to the whole trace
config = {
    'name': 'sequential chain',
    'tags': ['llm app', 'report generation', 'summarization'],
    'metadata': {'model1': "openai/gpt-oss-20b", 'model2': "openai/gpt-oss-20b",
'parser': 'StrOutputParser'}
}

result = chain.invoke({'topic': 'Unemployment in India'}, config = config)
```

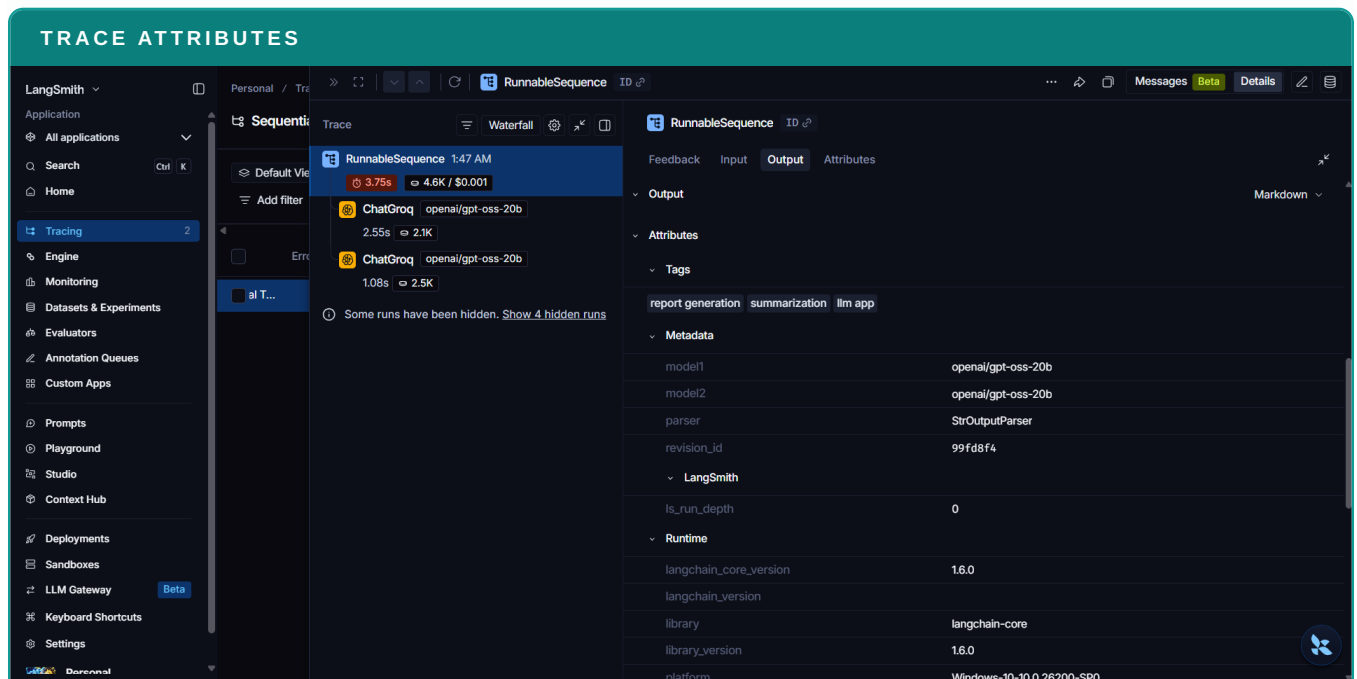
4.1 Setting the Project Name From Code

A project name can be set either in `.env` (as seen in Section 2) or directly inside the code with `os.environ['LANGCHAIN_PROJECT']`. When both are present, the value set in code takes precedence, because it is applied after `.env` is loaded. This makes it easy for a single codebase with multiple standalone scripts to route each script's traces to its own project.

4.2 Explicit Model Naming, Tags and Metadata

Using two separately instantiated `ChatGroq` models — `model1` for report generation, `model2` for summarization, each with its own temperature — makes each LLM run individually distinguishable in the trace, rather than relying on one shared model instance for two structurally different tasks.

Passing a `config` dictionary to `.invoke()` attaches free-form **tags** (a flat list of labels) and **metadata** (arbitrary key-value pairs) to the resulting trace. Both become searchable later: with a large number of traces accumulated over time, tags and metadata are what make a specific trace or a specific class of traces findable again.



The screenshot shows the LangSmith interface with the 'TRACE ATTRIBUTES' panel open for a 'RunnableSequence' trace. The left sidebar contains navigation options like 'All applications', 'Tracing', 'Engine', 'Monitoring', etc. The main panel displays the trace details, including a list of runs (e.g., ChatGPT, openai/gpt-oss-20b) and a detailed 'Attributes' section. The 'Attributes' section is divided into 'Tags' and 'Metadata'. The 'Tags' section lists 'report generation', 'summarization', and 'llm app'. The 'Metadata' section lists 'model1', 'model2', 'parser', 'revision_id', 'LangSmith' (with 'ls_run_depth' set to 0), and 'Runtime' (with 'langchain_core_version', 'langchain_version', 'library', 'library_version', and 'platform').

The trace's Attributes panel showing the custom tags (report generation, summarization, llm app) and metadata (model1, model2, parser) set via the config dictionary, alongside LangSmith's own auto-captured runtime metadata (library versions, platform).

Metadata and tags can also be set at the level of an individual run rather than the whole trace — for instance, each pipe-connected step is automatically tagged with its position in the sequence (`seq:step:1`, `seq:step:2`, and so on), which is how LangSmith knows to order and label steps consistently in the waterfall view.

4.3 Custom Run Names

By default, a LangChain sequential chain shows up in the trace list under the generic name `RunnableSequence`. Setting `run_name` inside the same `config` dictionary replaces that generic label with something meaningful, such as `sequential chain` — purely cosmetic, but valuable once a project accumulates many different chain shapes that would otherwise all be indistinguishable in a list of traces named `RunnableSequence`.



COMMON MISTAKE

It is easy to assume tags and metadata are purely cosmetic. In practice they are the primary mechanism for searching and filtering traces once a project accumulates thousands of executions — for example, filtering to only the traces that used a specific embedding model, to check whether a regression is isolated to that configuration.

Quick Revision — Sequential Chain Tracing

- `os.environ['LANGCHAIN_PROJECT']` set in code overrides the project name set in `.env`.
- `config = {'tags': [...], 'metadata': {...}, 'run_name': '...'}` passed to `.invoke()` attaches searchable labels to a trace.
- Each pipe-connected step in a sequential chain is auto-tagged with its position (`seq:step:N`).
- `run_name` replaces the generic 'RunnableSequence' label with a human-readable name in the trace list.

5. Tracing RAG Applications: From Partial Traces to a Cached Pipeline

The RAG demonstration is treated in four progressive versions. Each version fixes exactly one problem exposed by tracing the previous version — which is itself the central lesson of this section: **observability does not just show you a working system, it actively surfaces the specific defects that need fixing next.**

The application queries a machine-learning textbook (*Introduction to Statistical Learning*, as a local PDF) with questions like "Who is the author of this book?" using a standard retrieve-then-generate architecture:

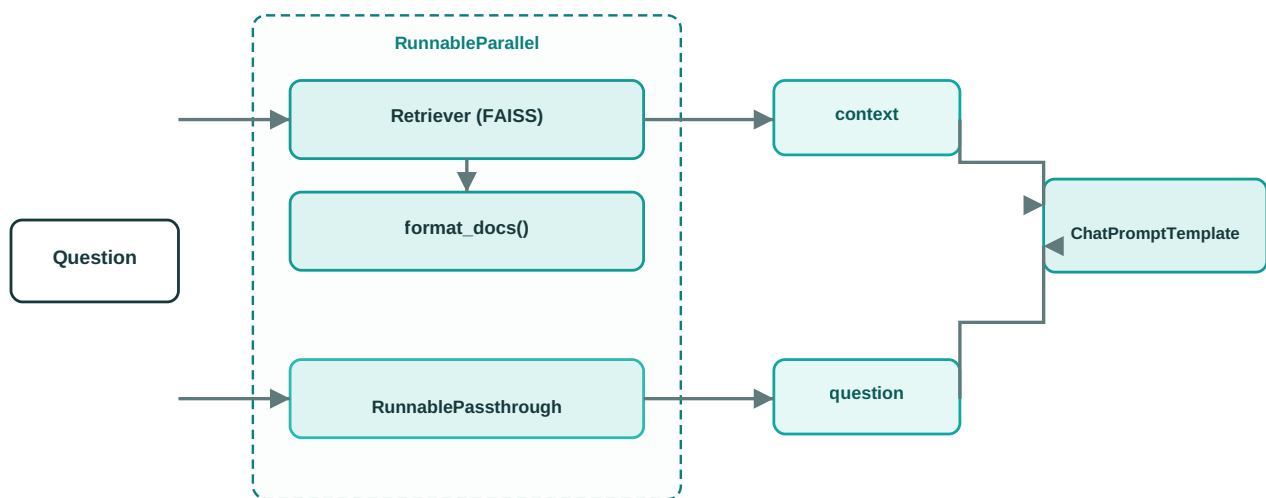


Figure 4.1 — The RAG chain fans the question into a retrieval path and a passthrough path, then merges both into the prompt.

5.1 RAG v1 — A Correct but Only Partially Traced Chain

The first version loads the PDF, chunks it, embeds the chunks into a FAISS vector store, and assembles a standard LangChain RAG chain — a `RunnableParallel` that produces `context` (via the retriever) and `question` (via passthrough) side by side, feeding into a prompt, an LLM, and a string parser.

```

loader = PyPDFLoader(PDF_PATH)
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150)
splits = splitter.split_documents(docs)

emb = HuggingFaceEmbeddings(model_name="BAAI/bge-small-en-v1.5")
vs = FAISS.from_documents(splits, emb)
retriever = vs.as_retriever(search_type="similarity", search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the provided context. If not found, say you don't know."),
    ("human", "Question: {question}\n\nContext:\n{context}")
])

llm = ChatGroq(model="openai/gpt-oss-20b", temperature=0)
def format_docs(docs): return "\n\n".join(d.page_content for d in docs)

parallel = RunnableParallel({
    "context": retriever | RunnableLambda(format_docs),
    "question": RunnablePassthrough()
})

chain = parallel | prompt | llm | StrOutputParser()
ans = chain.invoke(q.strip())

```

Because `retriever`, the parallel block, the prompt, the LLM and the parser are all LangChain runnables, invoking the chain is automatically traced — including the retriever's own run, which is exactly the visibility Scenario Three (Section 1.3) was missing.

TRACE SNAPSHOT — RETRIEVER RUN

The screenshot displays the LangChain trace viewer interface. On the left, a sidebar shows the project structure with 'RAG Chatbot V1' selected. The main area is titled 'TRACE SNAPSHOT — RETRIEVER RUN'. It features a 'Trace' panel showing a sequence of runs: 'RunnableSequence' (1:34 AM), 'map[key:context]' (0.05s), 'VectorStoreRetriever' (0.05s), 'format_docs' (0.00s), and 'ChatGroq' (openai/gpt-oss-20b, 1.21s). The 'VectorStoreRetriever' run is highlighted. To the right, the 'Details' panel for the 'VectorStoreRetriever' run is shown. It includes a 'Feedback' section and an 'Input' section with the query 'Who is the author of this book?'. The 'Output' section shows a list of two document objects, each with an ID, page_content, type, and metadata.

The `VectorStoreRetriever` run inside the v1 trace. Input: the raw question string. Output: the actual retrieved Document objects, each with its `page_content` and `metadata` — exactly the evidence needed to check whether the retriever found the right chunks.

The screenshot shows the LangSmith interface for tracing a 'RAG Chatbot V1' run. The main panel displays a 'TRACE SNAPSHOT — FINAL LLM CALL' for a 'RunnableSequence'. The trace shows the following steps:

- RunnableSequence** (1:34 AM): Total cost 1.37s, 1.458K / \$0.0003.
- map:key=context** (0.05s)
- VectorStoreRetriever** (0.05s)
- format_docs** (0.00s)
- ChatGroq** (openai/gpt-oss-20b): 1.21s, 1.458K.

The ChatGroq step's input is shown in the right panel, featuring a system message: "Answer ONLY from the provided context. If not found, say you don't know." followed by a user question: "Who is the author of this book?" and a retrieved context block containing text about Trevor Hastie and Robert Tibshirani.

The ChatGroq run's input, downstream of the retriever: the system message enforcing context-only answering, followed by the exact assembled human message containing both the question and the merged retrieved context. This is the precise text the model actually saw — the second place a RAG hallucination can be diagnosed.

Between these two runs, both classes of RAG failure described in Section 1.3 become directly checkable: a wrong answer can now be traced either to the retriever run (wrong or irrelevant chunks) or to the assembled prompt seen by the LLM (correct chunks, but a weak instruction or a model that ignored them).

COMMON MISTAKE

Running this v1 script a second time re-loads the PDF, re-chunks it and re-embeds every chunk from scratch, even though nothing about the document changed. This is invisible in the trace because loading, chunking and embedding happen as plain Python calls, outside any LangChain runnable — LangSmith only auto-traces runnable `.invoke()` calls.

5.2 RAG v2 — @traceable Fills the Gap, But Creates Two Disconnected Traces

LangSmith's `traceable` decorator can wrap *any* Python function — not just LangChain runnables — turning it into a run with its own inputs, outputs and latency. Version 2 wraps `load_pdf`, `split_documents` and `build_vectorstore` individually, and wraps all three again inside a parent `setup_pipeline` function, so the previously invisible indexing stage now appears in LangSmith too.

```

from langsmith import traceable # key import

@traceable(name = 'load_pdf')
def load_pdf(path: str):
    loader = PyPDFLoader(path)
    return loader.load()

@traceable(name = 'split_documents')
def split_documents(docs, chunk_size = 1000, chunk_overlap = 150):
    splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
chunk_overlap=chunk_overlap)
    return splitter.split_documents(docs)

@traceable(name = 'build_vectorstore')
def build_vectorstore(splits):
    emb = HuggingFaceEmbeddings(model_name="BAAI/bge-small-en-v1.5")
    return FAISS.from_documents(splits, emb)

@traceable(name = 'setup_pipeline')
def setup_pipeline(pdf_path: str):
    docs = load_pdf(pdf_path)
    splits = split_documents(docs)
    vs = build_vectorstore(splits)
    return vs

# Built once, at import time — a separate top-level trace
vectorstore = setup_pipeline(PDF_PATH)
retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 4})
# ... parallel / prompt / llm / parser exactly as in v1 ...

config = {'run_name': 'pdf_rag_query'}
ans = chain.invoke(q, config) # a second, separate top-level trace

```

TRACING PROJECT — RAG CHATBOT V2

Personal / Tracing / RAG Chatbot V2	RAG Chatbot V2	Tracing	Evaluators	Automations	Insights	Engine	New	Retention 14d	Dashboard	+ New
Default View	Last 1 day	Threads	Traces	Runs	Input	Search input content				
Add filter										
	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	
☒	pdf_rag_query	What is the full form of G...	GLM stands for **General...		9/3/2026, 1:44...	0.61s			1.01K	
☒	setup_pipeline	islr.pdf	<langchain_community.v...		9/3/2026, 1:42...	94.94s				

Two separate top-level rows in the same project: `setup_pipeline` (94.94s — loading, chunking, embedding) and `pdf_rag_query` (0.61s — the retrieval-and-generation chain). They are two independent traces, not one nested tree.

This introduces a new, purely structural problem: `setup_pipeline(...)` is called at import time, outside the traced chain invocation, so it becomes its own root trace — completely disconnected from the `pdf_rag_query` trace produced by the later `chain.invoke(...)` call. A single logical request (index the document, then answer a question about it) is now split across two unrelated trace rows, which defeats the purpose of having one coherent end-to-end view.

COMMON MISTAKE

A second, independent problem also remains in v2: nothing here changes the fact that `setup_pipeline` is called once per script run and always rebuilds the index from scratch. Fixing the trace hierarchy (Section 5.3) and fixing the rebuild-every-time latency problem (Section 5.4) are two separate engineering tasks, easy to conflate but requiring different fixes.

5.3 RAG v3 — One Root Trace via a Wrapping `@traceable` Function

Version 3 fixes the structural problem by introducing one additional `@traceable` function, `setup_pipeline_and_query`, that calls both `setup_pipeline(...)` and `chain.invoke(...)` from inside itself. Because both calls now happen inside the same traced function body, LangSmith nests both of them as children of one shared root run.

```
@traceable(name="setup_pipeline", tags=["setup"])
def setup_pipeline(pdf_path, chunk_size=1000, chunk_overlap=150):
    docs = load_pdf(pdf_path)
    splits = split_documents(docs, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    vs = build_vectorstore(splits)
    return vs

# One root traceable function wraps BOTH setup and query
@traceable(name="pdf_rag_full_run")
def setup_pipeline_and_query(pdf_path: str, question: str):
    vectorstore = setup_pipeline(pdf_path, chunk_size=1000, chunk_overlap=150)
    retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 4})

    parallel = RunnableParallel({
        "context": retriever | RunnableLambda(format_docs),
        "question": RunnablePassthrough(),
    })
    chain = parallel | prompt | llm | StrOutputParser()

    lc_config = {"run_name": "pdf_rag_query"}
    return chain.invoke(question, config=lc_config) # nests under the SAME root now
```

TRACING PROJECT — RAG CHATBOT V3

LangSmith

Application

All applications

Search

Home

Tracing

Engine

Monitoring

Datasets & Experiments

Evaluators

Annotation Queues

Custom Apps

Prompts

Playground

Studio

Context Hub

Deployments

Sandboxes

Personal / Tracing / RAG Chatbot V3

RAG Chatbot V3

ID

Tracing

Evaluators

Automations

Insights

Engine

New

Retention 14d

Dashboard

+ New

Default View

Last 1 day

Threads

Traces

Runs

Input

Search input content

Add filter

</>

	Name	Input	Output	Error	Start Time	Latency	Dataset
☒	pdf_rag_full_run	islr.pdf	Linear regression is a sup...		9/3/2026, 2:12:...	96.82s	

A single top-level trace, `pdf_rag_full_run` (96.82s), replacing the two disconnected rows seen in v2. The indexing work and the query work are now provably part of the same logical execution.

IMPORTANT

The rule that governs LangSmith's nesting is simple and mechanical: a run becomes a *child* of whichever traced function or runnable was already executing when it was called. Placing two calls inside the same `@traceable` parent function is therefore sufficient, by itself, to merge them into one trace tree — no special linking API is needed.

Version 3 still rebuilds the FAISS index from scratch on every single invocation — the 96.82s latency is barely different from v2's setup time. The trace hierarchy is now correct; the performance problem is not yet solved.

5.4 RAG v4 — Caching the Index With a Content Fingerprint

Version 4 keeps the same unified `pdf_rag_full_run` root, but replaces the unconditional "always rebuild" logic with a fingerprint-based cache: a SHA-256 hash of the PDF's contents, size and modification time, combined with the chunk size, chunk overlap and embedding model name, becomes a cache key. If a FAISS index already exists on disk under that exact key, it is loaded; otherwise it is built once and saved for next time.

Chapter 1 · LangSmith — 15

```

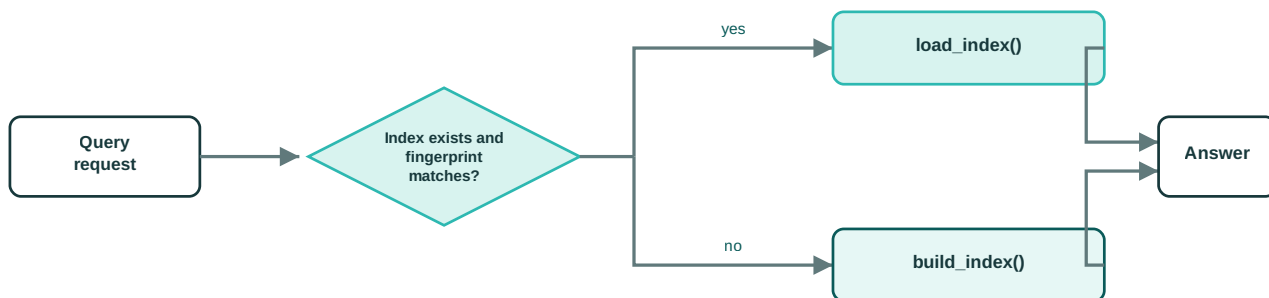
def _index_key(pdf_path, chunk_size, chunk_overlap, embed_model_name):
    meta = {
        "pdf_fingerprint": _file_fingerprint(pdf_path),    # sha256 + size + mtime
        "chunk_size": chunk_size, "chunk_overlap": chunk_overlap,
        "embedding_model": embed_model_name,
    }
    return hashlib.sha256(json.dumps(meta, sort_keys=True).encode()).hexdigest()

@traceable(name="load_index", tags=["index"])
def load_index_run(index_dir, embed_model_name):
    emb = HuggingFaceEmbeddings(model_name="BAAI/bge-small-en-v1.5")
    return FAISS.load_local(str(index_dir), emb, allow_dangerous_deserialization=True)

@traceable(name="build_index", tags=["index"])
def build_index_run(pdf_path, index_dir, chunk_size, chunk_overlap, embed_model_name):
    docs = load_pdf(pdf_path)
    splits = split_documents(docs, chunk_size=chunk_size, chunk_overlap=chunk_overlap)
    vs = build_vectorstore(splits, embed_model_name)
    index_dir.mkdir(parents=True, exist_ok=True)
    vs.save_local(str(index_dir))
    return vs

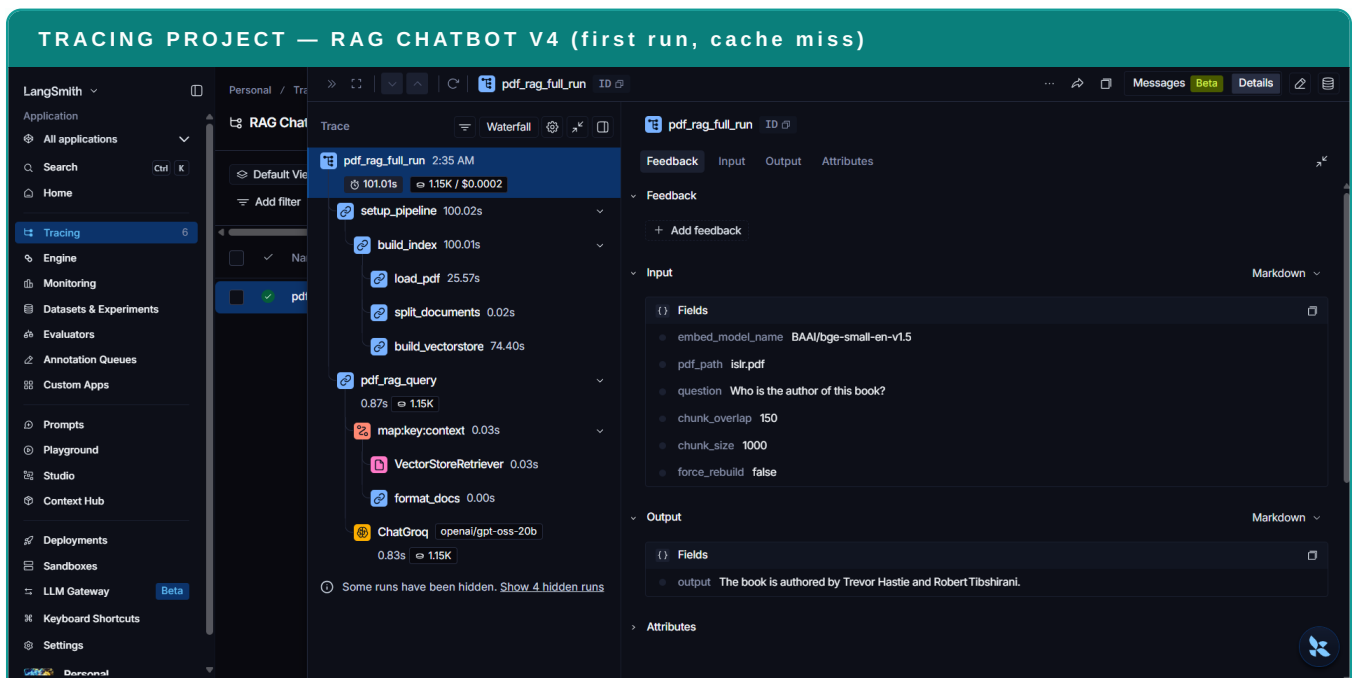
def load_or_build_index(pdf_path, chunk_size=1000, chunk_overlap=150,
                        embed_model_name="BAAI/bge-small-en-v1.5", force_rebuild=False):
    key = _index_key(pdf_path, chunk_size, chunk_overlap, embed_model_name)
    index_dir = INDEX_ROOT / key
    if index_dir.exists() and not force_rebuild:
        return load_index_run(index_dir, embed_model_name)    # cache hit → fast
    else:
        return build_index_run(pdf_path, index_dir, chunk_size, chunk_overlap, embed_model_name)

```



Rebuild triggers: new/changed PDF, changed chunk size or overlap, changed embedding model.

Figure 4.3 — RAG v4 checks a content fingerprint before deciding whether to reuse a saved FAISS index or rebuild it.



The full, correctly nested hierarchy on the very first run, when no cached index exists yet: `pdf_rag_full_run` → `setup_pipeline` → `build_index` → `load_pdf` (25.57s), `split_documents` (0.02s), `build_vectorstore` (74.40s); and, side by side, `pdf_rag_query` → the retrieval-parallel block → `ChatGroq`.

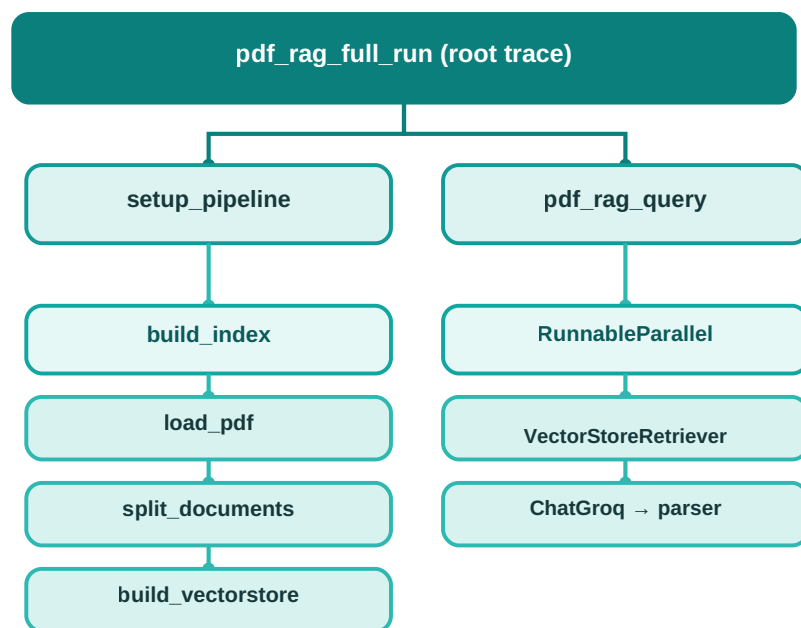


Figure 4.2 — The corrected RAG trace: one root run containing both the indexing pipeline and the query pipeline as children.

On this first run the total latency (101.01s) is dominated by `build_vectorstore`, since embeddings must be computed for all 441 pages of the source PDF. On every subsequent run with the same PDF, chunk settings and embedding model, `load_or_build_index` takes the cache-hit branch: `build_index` is replaced by a fast `load_index` run that reads the saved FAISS index straight from disk, and the described behavior (per the lecture) is a full round trip in roughly one to five seconds instead of over a minute.

BEST PRACTICE

Whenever a RAG pipeline is deployed to production, persist the vector index and key it by a fingerprint of everything that affects it (source document, chunk size, chunk overlap, embedding model). Rebuilding embeddings from scratch on every request is the single most common avoidable latency cost in a RAG system.

Version	What changed	Trace structure	Rebuild behavior
v1	Plain LangChain RAG chain	Chain traced; PDF load/chunk/embed invisible	Every run — implicitly
v2	@traceable on setup functions	Two disconnected top-level traces	Every run — now visible, still wasteful
v3	One wrapping @traceable root	Single unified trace tree	Every run — still wasteful
v4	Fingerprinted FAISS cache	Single unified trace tree	Only on cache miss

Quick Revision — RAG Tracing Evolution

- LangSmith auto-traces LangChain runnables only; plain Python function calls (like PDF loading) need the @traceable decorator to appear in a trace.
- A run becomes a child of whichever traced function or runnable was already executing when it was invoked — nesting is purely about call structure, not a special API.
- Two separately traced top-level calls (e.g. setup at import time, query later) produce two disconnected traces even if they logically belong together.
- Wrapping both calls inside one shared @traceable parent function merges them into a single root trace.
- A content-fingerprint cache (hash of file + chunk settings + embedding model) avoids rebuilding a vector index on every run — the trace makes both the problem and the fix directly measurable.

6. Tracing Agents: The Tool-Calling Loop

Section 1.2 described why an autonomous agent is the hardest of the three failure scenarios to debug: it reasons, acts and re-evaluates itself in a loop, so the same kind of prompt change that seems harmless can alter cost or behavior only for some inputs. This section traces a small agent equipped with two tools — a DuckDuckGo web search and a custom weather-lookup function — to see exactly what that loop looks like inside LangSmith.

```

search_tool = DuckDuckGoSearchRun()

@tool
def get_weather_data(city: str) -> str:
    """Fetches the current weather data for a given city."""
    api_key = os.getenv("WEATHER_STACK_API_KEY")
    url = f"https://api.weatherstack.com/current?access_key={api_key}&query={city}"
    return requests.get(url).json()

llm = ChatGroq(model="openai/gpt-oss-20b")

# Native tool-calling agent (LangGraph-based) – NOT the classic ReAct agent
agent = create_agent(
    model=llm,
    tools=[search_tool, get_weather_data],
    system_prompt="You are a helpful AI assistant. Use the available tools when necessary."
)

response = agent.invoke({
    "messages": [{"role": "user", "content": "What is the current temp of gurgaon"}]
})

```

COMMON MISTAKE — LECTURE VS. ATTACHED CODE

The lecture narrates the classic **ReAct** agent pattern in detail: a hand-written prompt instructs the model to emit **Thought / Action / Observation** text, which is manually appended to a growing "agent scratchpad" and re-sent to the LLM on every loop iteration, built with LangChain's older `create_react_agent`. However, the attached source file (`4_agent.py`) and its actual LangSmith trace use LangChain's newer `create_agent` API, which relies on the model's **native tool-calling** capability instead of a text-based scratchpad. This is a genuine mismatch between the spoken walkthrough and the real implementation, not a simplification — the two produce visibly different trace shapes, and only the native tool-calling shape is shown below because that is what the code and screenshot actually demonstrate.

With native tool calling, the loop has three participants instead of a scratchpad: the **model**, which decides whether it has enough information to answer or needs a tool; the **tools** node, which executes whichever tool the model requested; and the loop edge that feeds the tool's result back into the model for another decision.

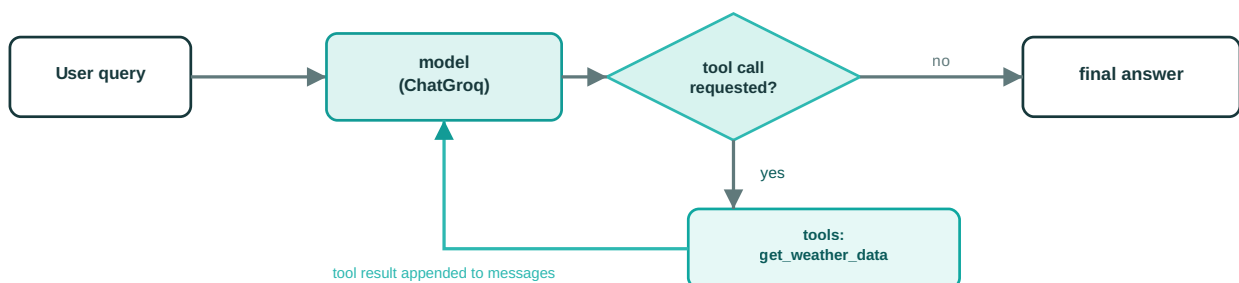
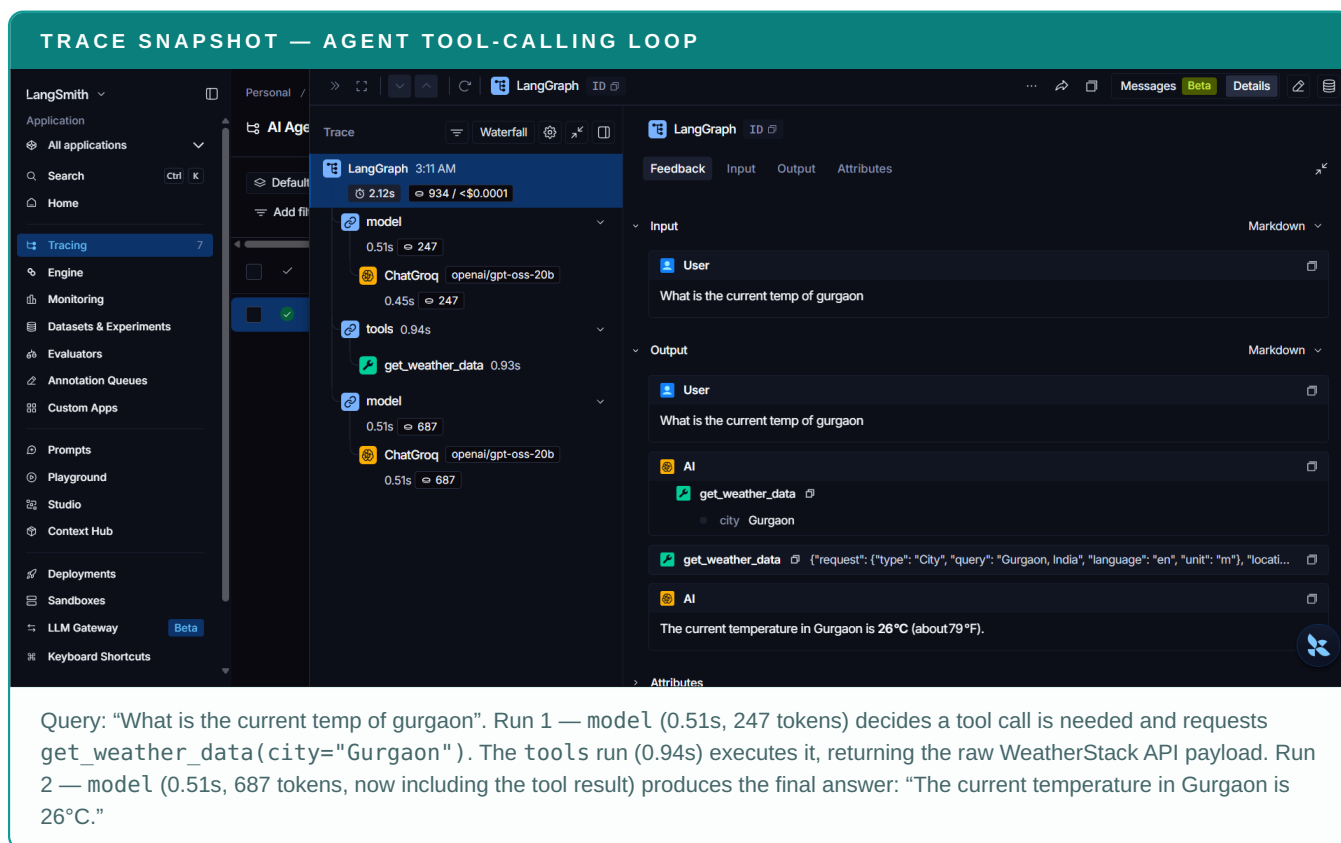


Figure 5.1 — The tool-calling agent loop: the model calls a tool, the tool result feeds back into the model, and the loop repeats until a final answer is ready.



Every element needed to debug this loop is directly visible: exactly which tool was called and with what argument, the raw data the tool returned, how many tokens each model call consumed, and the latency of each step (2.12s total, of which 0.94s was the external weather API call). If the agent had instead called the search tool unnecessarily, or called a tool with a malformed argument, that would appear directly as an extra, inspectable run rather than as an unexplained change in total cost or latency.

6.1 Chaining Two Different Tools in One Loop

The lecture also runs a compound query through this same agent: *"Identify the birthplace city of Kalpana Chawla, and give its current temperature."* Answering this requires the agent to use **two different tools in sequence** within a single tool-calling loop — the search tool first, to establish that Kalpana Chawla was born in Karnal, and only then the weather tool, using Karnal as the argument derived from the search result rather than from the user's original question.

REAL WORLD INSIGHT

This is precisely the kind of multi-step reasoning that makes agents powerful and simultaneously hard to debug without tracing: the input to the second tool call (`city="Karnal"`) does not appear anywhere in the user's original question — it is a value the model derived from the first tool's result. A trace makes that derivation step directly visible; without it, "why did the agent look up the weather for Karnal?" would be unanswerable from the final output alone.

Structurally, this longer query simply extends the same model → tools → model loop from Figure 5.1 by one more iteration: model decides to search, tools run the search, model reads the search result and decides to call the weather tool, tools run the weather lookup, and the model produces the final answer — five runs total inside one trace, each independently inspectable exactly as in the single-tool case.

Quick Revision — Agent Tracing

- The lecture narrates the classic ReAct (Thought/Action/Observation scratchpad) pattern, but the attached code and its trace actually use the newer native tool-calling `create_agent` API — a real discrepancy worth remembering.
- A native tool-calling agent trace alternates model runs and tools runs: the model decides whether to call a tool, the tools run executes it, and the result feeds back into the next model run.
- Each loop iteration is independently inspectable — which tool was called, with what input, what it returned, and how many tokens and how much latency that step cost.
- This directly solves the Scenario Two cost-debugging problem: a spike in cost now traces to a specific extra loop iteration rather than remaining a mystery.
- Multi-tool queries simply extend the same loop by more iterations — each additional tool call is one more inspectable model/tools run pair inside the same trace.

7. LangSmith + LangGraph Integration

LangGraph and LangSmith are built by the same team and integrate tightly, but the mental model needed to read a LangGraph trace only requires two rules, extending the Project/Trace/Run vocabulary from Section 2:

- Executing a compiled graph from **START** to **END** becomes **one trace** in LangSmith — exactly as a chain invocation does.
- Every **node** the graph passes through becomes **one run** inside that trace, and any branching, parallel fan-out or looping in the graph is reflected directly in the run hierarchy.

The example graph evaluates a UPSC-style essay along three independent dimensions — language, depth of analysis, and clarity of thought — in parallel, then combines all three into one overall verdict.

```

class UPSCState(TypedDict, total=False):
    essay: str
    language_feedback: str
    # ... analysis_feedback, clarity_feedback, overall_feedback: str ...
    individual_scores: Annotated[List[int], operator.add] # merges parallel results
    avg_score: float

# Each node function is ALSO wrapped in @traceable – optional, but gives
# a nested function-level run inside the node-level run LangGraph already creates.
@traceable(name="evaluate_language_fn", tags=["dimension:language"])
def evaluate_language(state: UPSCState):
    prompt = "Evaluate the language quality...\n\n" + state["essay"]
    out = structured_model.invoke(prompt)
    return {"language_feedback": out.feedback, "individual_scores": [out.score]}

# ... evaluate_analysis, evaluate_thought defined the same way ...

@traceable(name="final_evaluation_fn", tags=["aggregate"])
def final_evaluation(state: UPSCState):
    overall = model.invoke("Based on the following feedback...").content
    scores = state.get("individual_scores", []) or []
    avg = sum(scores) / len(scores) if scores else 0.0
    return {"overall_feedback": overall, "avg_score": avg}

graph = StateGraph(UPSCState)
graph.add_node("evaluate_language", evaluate_language)
# ... add_node for evaluate_analysis, evaluate_thought, final_evaluation ...

# Fan-out from START, fan-in to final_evaluation
graph.add_edge(START, "evaluate_language")
graph.add_edge(START, "evaluate_analysis")
graph.add_edge(START, "evaluate_thought")
graph.add_edge("evaluate_language", "final_evaluation")
graph.add_edge("evaluate_analysis", "final_evaluation")
graph.add_edge("evaluate_thought", "final_evaluation")
graph.add_edge("final_evaluation", END)

workflow = graph.compile()
result = workflow.invoke({"essay": essay2}, config={"run_name": "evaluate_upsc_essay"})

```

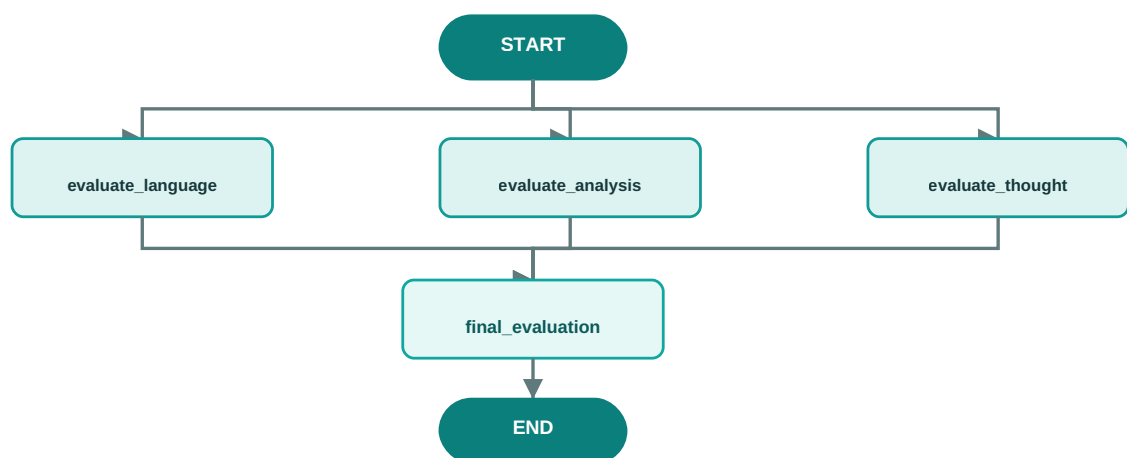
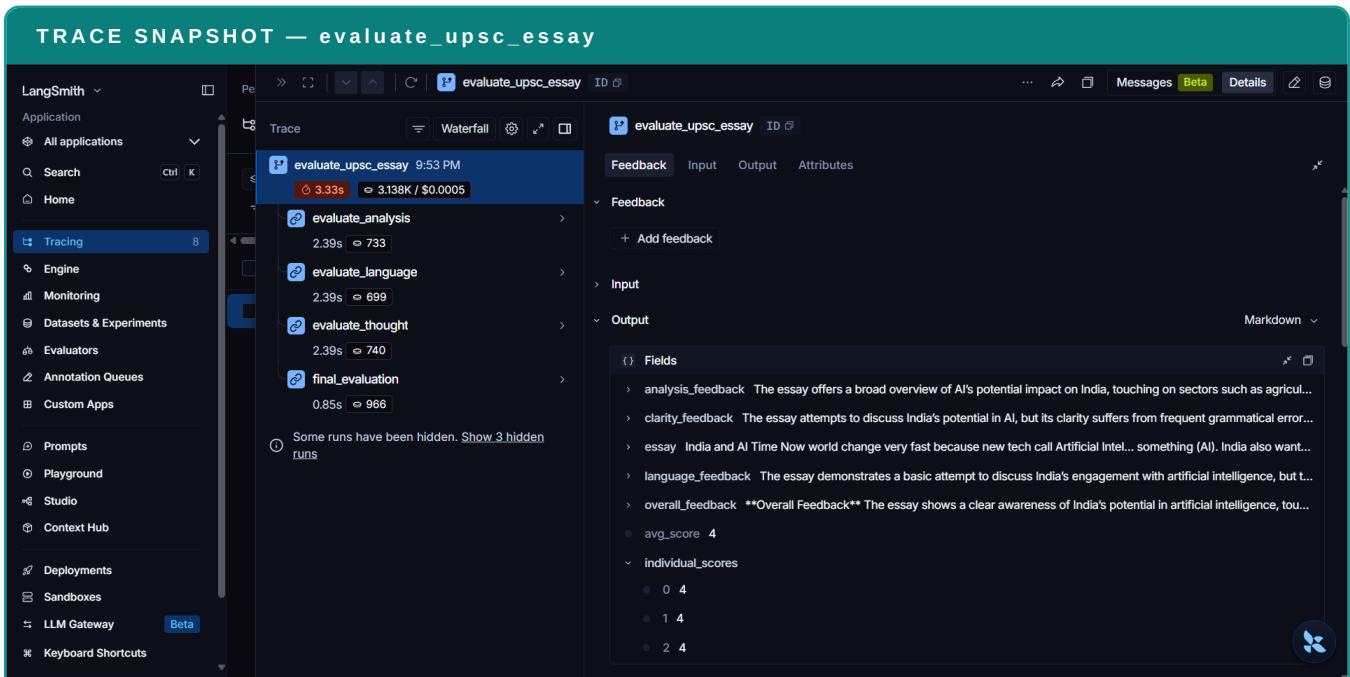


Figure 6.1 — LangGraph traces the fan-out into three parallel evaluator nodes and the fan-in into one final node as a single graph execution.



The custom run name (set via `run_name` in config) replacing the default graph name. Three parallel node runs — `evaluate_analysis` (2.39s), `evaluate_language` (2.39s), `evaluate_thought` (2.39s) — followed by `final_evaluation` (0.85s). The Feedback tab shows the fully structured output: per-dimension feedback text, an `avg_score` of 4, and the three `individual_scores` merged via the state's operator `.add reducer`.

Because each node function is additionally decorated with `@traceable`, every node run contains one nested function-level run with the same name plus an `_fn` suffix — this is optional (LangGraph nodes are traced automatically regardless), but it lets a developer distinguish "the node executed" from "the underlying function's own inputs and outputs," which becomes useful once a node's internal logic grows more complex than a single LLM call.

REAL WORLD INSIGHT

Each evaluator node uses `model.with_structured_output(EvaluationSchema)` so the LLM's reply is coerced into a validated `{feedback, score}` object. Inside the trace, this shows up as an additional nested `RunnableSequence` wrapping the raw LLM call together with a `RunnableLambda` that parses the structured response — the same "structured output" internals appear in any chain that uses this feature, not just in LangGraph.

7.1 Why LangGraph Needs LangSmith Even More Than an Agent Does

An agent built with `create_agent` (Section 6) decides its own control flow at runtime — the loop shape is not fixed in advance and only becomes apparent from the trace after execution. A LangGraph workflow is the opposite: its nodes and edges are declared explicitly in code before anything runs, so the developer already knows the possible shape of execution. This might suggest LangGraph traces are less necessary, but in practice the opposite is true: as soon as a graph grows past a handful of nodes — with conditional edges, parallel branches, and sub-graphs — the sheer number of possible paths through the declared structure becomes hard to hold in one's head, and confirming which path was actually taken on a given run again requires a trace. LangSmith and LangGraph being built by the same team is precisely why this integration needs no extra setup beyond what is already used for LangChain.

7.2 The Complexity Ladder: Comparing All Five Traced Applications

Every application traced in this chapter is traced using the exact same two ideas — one execution is one trace, one component is one run — regardless of how simple or elaborate the application becomes. Laid side by side, the five demonstrations form a single progression from the simplest possible LLM call to a full graph workflow:

Application	Main execution pattern	Typical traced components	What becomes visible
Simple LLM call	Single model call	Prompt run, LLM run, parser run	Exact input/output and latency of one model call
Sequential chain	Multiple dependent calls	Chain + per-step child runs, custom tags/metadata	Step-by-step execution order and per-step timing
RAG	Retrieval + generation	Retriever run, format step, prompt, LLM run	Retrieved context and the exact assembled prompt
Agent	Reasoning + tool use, in a loop	Alternating model runs and tools runs	Which tools were called, with what arguments, in what order
LangGraph	Declared graph execution	One run per node, including parallel/branching runs	Which path through the declared graph was actually taken

This progression is also a progression in debugging difficulty: a simple call rarely needs tracing at all, while an agent or a graph is nearly impossible to debug reliably without it — which is exactly the arc this chapter has followed, from Section 1's three untraceable failures to a fully traced graph workflow in Section 7.

Quick Revision — LangGraph Tracing

- One full graph execution (START to END) becomes one trace.
- Every node the graph visits becomes one run inside that trace; parallel fan-out and fan-in in the graph appear directly as parallel and converging runs.
- Wrapping a node's underlying function in `@traceable` adds an optional nested function-level run for finer granularity.
- `with_structured_output` produces a nested `RunnableSequence` + `RunnableLambda` inside the run, visible in any structured-output call — not LangGraph-specific.
- `run_name` in the invoke config renames the graph's root trace exactly as it does for a plain chain.
- Across all five demonstrations in this chapter, only two ideas ever change scale: one execution = one trace, one component = one run.

8. Beyond Observability: The Rest of the LLMOps Toolkit

Everything so far has been a single perspective on LangSmith: observability, meaning the ability to study one trace in detail. LangSmith also supports several other capabilities that together are usually grouped under the broader umbrella term **LLMOps** — running an LLM application reliably in production, not just building it once.

8.1 Monitoring & Alerting

Where observability studies *one* trace, monitoring looks across *many* traces at once to track the overall health of a system: it aggregates metrics like latency, token usage, cost, error rate and success rate over a chosen time window, and renders them as dashboards (traces per day, latency trend, cost per trace, and so on). Alerting builds on top of monitoring — a threshold on a chosen metric (for example, "latency greater than 5 seconds") triggers a notification the moment it is crossed, rather than waiting for a customer complaint.

REAL WORLD INSIGHT

Production issues often first appear as a *pattern across many runs* rather than as a single broken trace. Monitoring is what catches a slow upward drift in latency or cost early, before it affects users at scale — observability alone, focused on one trace at a time, would not surface a trend like that.

8.2 Evaluation

Because LLM behavior is probabilistic, a small change — a different prompt, a different model, a different retriever configuration — can improve some inputs while quietly breaking others. Evaluation gives an objective, repeatable way to measure whether a new version is actually better: a set of test inputs is run through the application, each output is scored against some criteria, and the aggregate score is compared across versions before anything is deployed.

LangSmith supports several evaluation approaches: automated scoring with an LLM acting as a judge, semantic similarity checks against a reference answer, and fully custom Python evaluator functions scored against criteria such as faithfulness, relevance and completeness. Built-in evaluators are also available for common failure modes such as hallucination and overly verbose (non-concise) responses, so a team does not need to write a custom scorer for the most frequent failure patterns.

8.3 Prompt Experimentation

Since prompt wording measurably affects output quality, LangSmith's Playground supports structured A/B testing of prompts: two or more prompt variants can be run against the same dataset, scored against the same evaluation criteria, and compared side by side — including testing the same prompt across different candidate models to see which one performs best for a given task. Results are stored over time, building a history of which prompt variation performed best under which conditions, so a regression introduced by a "small" prompt edit (echoing Scenario Two in Section 1.2) can be caught before it reaches production rather than after.

8.4 Dataset Creation & Annotation

Evaluation requires a dataset of representative inputs (and, optionally, expected outputs) to test against. LangSmith lets a team build such a dataset from scratch, import existing rows, or promote any real trace directly into a dataset with a single click — turning a genuine production example (including edge cases discovered through observability) directly into a permanent regression test. Individual examples can then be manually annotated or labeled — for example, marking whether a given answer was factually correct — typically through a dedicated annotation queue, so that a team can systematically work through a backlog of examples needing review.

8.5 User Feedback Integration

Just as a thumbs-up / thumbs-down control lets end users rate a chatbot's reply, LangSmith exposes an API for attaching that same kind of feedback directly onto a trace. Feedback recorded this way is tied to the exact prompt, model and state that produced the response, and feeds directly into the monitoring dashboards described in Section 8.1 — so a sustained dip in user sentiment becomes just another metric that can be tracked over time and, if needed, alerted on.

8.6 Collaboration

Before a shared tracing tool, debugging a production issue across a team typically meant emailing screenshots back and forth. LangSmith instead lets any trace be shared via a direct link, so a teammate opens the exact same trace, with the exact same data, on their own machine. The same collaborative model extends to prompts (shared, versioned, commented on) and to custom dashboards, which can also be shared across a team.

Capability	Answers the question	Operates on
Observability	What exactly happened in this one run?	A single trace
Monitoring & Alerting	Is the system healthy overall, right now?	Many traces, aggregated
Evaluation	Is version B actually better than version A?	A dataset, scored
Prompt Experimentation	Which prompt / model variant performs best?	Prompt variants on a dataset
Dataset & Annotation	What should "correct" look like?	Curated, labeled examples
User Feedback	Did the end user like this specific answer?	One trace, from the user's view
Collaboration	Can my teammate see exactly what I'm seeing?	Shared traces, prompts, dashboards

Quick Revision — Beyond Observability

- Monitoring aggregates metrics (latency, cost, tokens, error/success rate) across many traces; alerting fires a notification when a chosen metric crosses a threshold.
- Evaluation runs a dataset through the application and scores outputs objectively, so a new version can be compared against the old one before deployment.
- Prompt experimentation is essentially A/B testing for prompts and models against a shared dataset and evaluation criteria.
- Datasets can be built from scratch, imported, or created directly from real traces, then manually annotated for reuse.
- User feedback (thumbs up/down or structured ratings) attaches directly to the trace that produced the response.
- Collaboration features (shareable trace links, shared prompts, shared dashboards) replace ad-hoc screenshot-based debugging across a team.
- All of the above are commonly grouped under the umbrella term LLMOps.

K. Key Takeaways

- **Observability exists because LLM systems are non-deterministic black boxes.** Latency regressions, cost spikes and hallucinations all share the same root difficulty: without per-component visibility, a symptom cannot be traced to its cause.
- **Project → Trace → Run is the entire vocabulary.** A project is the application; each execution of it is a trace; each component inside that execution is a run.
- **LangChain runnables are traced automatically; plain functions need @traceable.** Nesting is determined purely by call structure — whichever traced function or runnable was already running when another one is called becomes its parent.
- **The RAG evolution (v1–v4) is a template for real debugging work:** fix visibility first (traceable decorators), then fix trace structure (one shared root), then fix performance (fingerprinted caching) — three distinct problems, three distinct fixes.
- **Agents and LangGraph both reduce to the same two ideas:** one execution = one trace; one component (tool call, node) = one run — regardless of how deep the loop or how wide the fan-out.
- **Tags, metadata and custom run names** are what make individual traces findable again once a project accumulates thousands of executions.

- **Observability is only one of several LangSmith capabilities.** Monitoring, evaluation, prompt experimentation, dataset curation, user feedback and collaboration together form the broader LLMOps toolkit needed to run an LLM application reliably, not just build it once.

R. Revision Sheet

Project / Trace / Run

Project = whole app. Trace = one execution. Run = one component's execution inside a trace. Strict one-way nesting.

Enabling tracing

`LANGCHAIN_TRACING_V2=true`,
`LANGCHAIN_ENDPOINT`, `LANGCHAIN_API_KEY`,
`LANGCHAIN_PROJECT`. No code changes needed for
LangChain runnables.

@traceable

Wraps any Python function into a run with its own input/output/latency, even if it contains no LangChain runnable.

Nesting rule

A run nests under whichever traced function/runnable was already executing when it was called — no explicit parent-linking API required.

Tags & metadata

Attached via a config dict passed to `.invoke()`, or as decorator arguments; make traces searchable at scale.

run_name

Overrides the generic auto-generated name (e.g. `RunnableSequence`) with a human-readable label for a trace or run.

RAG failure modes

Retriever error (wrong/irrelevant chunks) vs. generator error (LLM ignores correct context) — indistinguishable without a trace.

Index caching

Fingerprint = `hash(file contents + chunk size + chunk overlap + embedding model)`. Cache hit → `load_index`; cache miss → `build_index`.

Agent loop

model → decides on tool call → tools run executes it → result appended → back to model → repeat until final answer.

LangGraph tracing

One graph execution = one trace; one node = one run. Fan-out/fan-in and loops appear directly as parallel/converging runs.

Monitoring vs. Evaluation

Monitoring aggregates live metrics across many traces. Evaluation scores outputs against a dataset before deployment.

LLMOps umbrella

Observability + Monitoring/Alerting + Evaluation + Prompt Experimentation + Datasets/Annotation + Feedback + Collaboration.

I. Interview Questions

- 1 What is the difference between a project, a trace and a run in LangSmith, and how do they nest?
- 2 Why does a plain Python function not get traced automatically, and how does the `@traceable` decorator change that?
- 3 Given a RAG chatbot that hallucinates, how would you determine whether the fault lies in the retriever or the generator using only LangSmith trace data?
- 4 Explain the rule LangSmith uses to decide whether one run is nested under another as its child.
- 5 What structural problem occurs if a setup function and a query function are traced independently instead of from within a shared parent function?
- 6 Design a caching strategy for a RAG pipeline's vector index that avoids unnecessary rebuilds. What should the cache key include?
- 7 How does an agent's tool-calling loop appear in a LangSmith trace, and what would repeated unnecessary tool calls look like there?
- 8 How does LangSmith represent a LangGraph execution that fans out into parallel nodes and later joins them?
- 9 What is the difference between observability and monitoring, and why do you need both in production?
- 10 Why is evaluation necessary for LLM applications specifically, given that traditional software testing already exists?

F. Further Reading

- [LangSmith official documentation](#) — concepts, tracing, evaluation and deployment guides.
- [LangSmith Observability documentation](#) — tracing setup, the `@traceable` decorator, and run/trace structure in depth.
- [LangSmith Evaluation documentation](#) — datasets, evaluators, and LLM-as-judge scoring.
- [LangChain Python documentation](#) — Runnable interface, RunnableParallel, RunnablePassthrough and structured output.
- [LangGraph documentation](#) — StateGraph, nodes, edges, and the `create_agent` tool-calling API.